

BANDspirit

Application, frontend, database and documentation

An evidence-based register of implementation defects, integrity gaps, design decisions and documentation corrections. Each finding explains the consequence, the proposed repair and how to verify it.

69 review findings across 3 areas

22 application 32 frontend 15 database

Contents

1	Finding register	1
2	Application findings	3
3	Frontend, visual design and interaction findings	11
4	Database and migration findings	34

1 Finding register

The 69 entries include implementation defects, conditional risks, design decisions and documentation improvements. Related entries are cross-referenced because correcting a database shape does not automatically repair an API workflow. The count is not a count of reproduced failures, production incidents or evidence of how the code was authored. Reproduced, source-confirmed and design-improvement labels describe different levels of evidence.

ID	Priority	Finding
APP-01	High	Credential changes can enter the audit log
APP-02	High	Existing JWTs retain stale account and role state
APP-03	High	The visible logout button does not revoke the API token
APP-04	High	Redis failure removes the revocation guarantee
APP-05	High	Saving zero permissions silently restores fallback rights
APP-06	High	Photo replacement can delete the working image before validation
APP-07	Medium	Object storage and database changes lack durable compensation
APP-08	High	Password-reset consumption is not atomic
APP-09	High	Generic circle PATCH bypasses hierarchy invariants
APP-10	High	The dedicated hierarchy check uses an unlocked pre-transaction snapshot
APP-11	High	Lifecycle edits and deletions leave the displayed phase stale
APP-12	Medium	Permission cache invalidation reaches only one API process
APP-13	High	Open self-registration creates active accounts with directory access
APP-14	Medium	Dashboard and profiles issue avoidable sequential queries
APP-15	Medium	Profile lead-link display selects an arbitrary matching person
APP-16	Medium	Frontend session lifetime can diverge from API-token lifetime
APP-17	High	Single-holder role policy conflicts with the supplied import
APP-18	Medium	Client-supplied creator IDs can survive entity-bound create requests
APP-19	High	UTC date writes and log filters fail at the PostgreSQL boundary
APP-20	High	Updating an individual mailing list fails during membership replacement
APP-21	Medium	Single-role expansion loses assignments that collection expansion returns
APP-22	Medium	User administration accepts an invalid email address
UI-01	Medium	Help tabs retain stale state after sidebar navigation
UI-02	Low	Sidebar selection ignores tab query parameters
UI-03	Medium	The organigram extends beyond a phone-width viewport
UI-04	Medium	The mobile sidebar lacks a complete drawer interaction model
UI-05	Medium	Sidebar text contrast is too low for ordinary navigation labels
UI-06	High	Many pages send notifications to an unmounted toast system
UI-07	High	Unavailable data is presented as zero counts or no search results
UI-08	Medium	Several filters and search placeholders promise behavior not implemented
UI-09	Medium	Global search describes modules it never searches
UI-10	Low	Search results lead to whole lists instead of the matched item
UI-11	High	Support ticket rendering and editing assume undeclared backend fields
UI-12	High	The tension form posts English field names to a German model
UI-13	High	The OKR form collects owner, team and period without matching persistence fields
UI-14	High	Key Result ownership and numeric values use divergent names
UI-15	High	KPI editing assumes a richer model than the API declares
UI-16	Medium	Frontend lists do not follow server-driven OData pagination
UI-17	Medium	Circle summaries substitute zeros for data they have not loaded
UI-18	Medium	Application Log is hidden from the managed administrator navigation

Review register continued on next page

ID	Priority	Finding
UI-19	Medium	Application Log cannot filter the audit module the backend actually writes
UI-20	Medium	Icon actions and several form fields lack programmatic names
UI-21	Medium	FAQ cards nest edit and delete buttons inside another button
UI-22	Medium	Help dialogs have no explicit vertical overflow strategy
UI-23	Low	The dashboard gives limited direction when activity sections are empty
UI-24	Medium	The archive development proxy intercepts NextAuth routes
UI-25	Medium	Frontend action visibility uses incompatible permission models
UI-26	High	BI Compass crashes when rendering a persisted active version
UI-27	High	Circle editing submits a different contract from the persisted circle model
UI-28	High	Meeting creation and detail queries disagree with the API
UI-29	High	The user deactivation action deletes the account
UI-30	High	The lifecycle process overview calls an unimplemented endpoint
UI-31	High	Password recovery forms use incompatible request and response fields
UI-32	Medium	A direct user-detail visit can redirect before the session is ready
DB-01	High	Business identifiers lack referential integrity
DB-02	High	Application-role membership uses editable names
DB-03	High	Circle/definition uniqueness exists only in one create path
DB-04	High	Four detail tables allow zero or two owners
DB-05	High	The definition-owner migration does not backfill legacy details
DB-06	Medium	Responsibilities have three editable storage locations
DB-07	Medium	Repeated role names have no explicit template/variant semantics
DB-08	Medium	Validity fields do not define membership history
DB-09	High	VALIDATION PASSED covers fewer checks than the script prints
DB-10	High	Import transformations and onboarding decisions are not fully documented
DB-11	Medium	Organizational and global measurement models are disconnected
DB-12	Low	Company singleton exists only as an application convention
DB-13	Medium	Email comparison and canonicalization are not explicit
DB-14	Medium	Application startup performs schema migration
DB-15	Medium	Deletion and retention policies are implicit in cascades and cleanup

2 Application findings

APP-01 | Credential changes can enter the audit log

Priority: High. Confirmed code defect.

Observed behavior. The audit exclusion list names PasswordHash and TokenHash, while the actual properties are Password and Token. The UPDATE audit path serializes old and new property values.

Consequence. Password hashes can consequently be duplicated into AppLogs.Details and exposed to an additional audience or retained in extra backups.

Recommended correction. Use actual property identifiers or a sensitive-field allowlist. Record that a credential changed without its value; review existing audit data through an authorized process.

Acceptance check. Reset a synthetic account password and change it through user administration. Assert that neither old nor new hash appears in the audit record.

APP-02 | Existing JWTs retain stale account and role state

Priority: High. Reproduced account-state enforcement gap.

Observed behavior. The JWT validation event checks token revocation but does not reload the user or verify current Aktiv/Role. The general permission handler obtains the role from token claims.

Consequence. A previously issued token can continue through the general policy path after account deactivation or demotion, until expiry or separate revocation.

Recommended correction. Define a session invalidation contract. Check a current account/session version and activity state, and invalidate sessions after sensitive account changes.

Acceptance check. Keep a token from before deactivation, deletion, demotion and password reset. Each protected action must follow the agreed invalidation policy immediately, including on a second API instance.

APP-03 | The visible logout button does not revoke the API token

Priority: High. Confirmed integration gap.

Observed behavior. The dashboard logout button calls NextAuth signOut. The inspected NextAuth configuration contains no sign-out event that calls the backend logout endpoint, and the frontend source search found no /api/auth/logout call.

Consequence. Clearing the browser session is not equivalent to revoking an already issued backend bearer token.

Recommended correction. Wire logout to server-side backend revocation before clearing the session, with an explicit error policy.

Acceptance check. Sign in with a test account, save its token only in the test harness, log out using the actual button, and verify the token is rejected by the API.

APP-04 | Redis failure removes the revocation guarantee

Priority: High. Reproduced conditional revocation failure.

Observed behavior. RevokeAsync returns without persisting revocation when Redis is absent or disconnected and catches write errors. IsRevokedAsync returns false on unavailability or read errors.

Consequence. A logout response can imply revocation that did not occur.

Recommended correction. Document an approved failure policy. Use a dependable revocation/session store, propagate revocation failure, and consider rejecting sensitive requests when revocation cannot be checked.

Acceptance check. In a disposable environment, disconnect Redis during revocation and validation. Verify the response, token acceptance behavior, monitoring and recovery behavior against the chosen contract.

APP-05 | Saving zero permissions silently restores fallback rights

Priority: High. Reproduced permission fallback defect.

Observed behavior. The permission editor accepts an empty Permissions collection and removes existing rows. RbacService interprets zero database rows as missing setup and supplies fallback permissions.

Consequence. A non-admin role saved with no permissions receives eight fallback read permissions.

Recommended correction. Represent initialization separately from the permission collection. Empty must mean empty after configuration; unknown or inactive roles should fail closed.

Acceptance check. Save an empty permission set for a test role, clear/wait out caches, and assert every permission check returns false.

APP-06 | Photo replacement can delete the working image before validation

Priority: High. Confirmed operation-order defect.

Observed behavior. FotoHochladen deletes the old object before decoding the incoming image and before uploading or saving the replacement. An image/* content type passes the initial check even when image decoding later fails.

Consequence. A rejected image or failed replacement upload can leave the database pointing at a deleted old photo.

Recommended correction. Decode and validate first, upload under a new key, save the new database reference, then clean up the old object.

Acceptance check. Try invalid image bytes with an image content type, an upload failure, and a database-save failure. In each failed replacement, verify the previous photo remains retrievable.

APP-07 | Object storage and database changes lack durable compensation

Priority: Medium. Confirmed consistency gap.

Observed behavior. Document upload writes the object before inserting its metadata row. Delete attempts object removal and then deletes metadata, with some storage failures ignored.

Consequence. A database failure after upload can leave an orphan object; a delete failure can leave bytes without metadata.

Recommended correction. Use explicit operation states and an outbox or durable cleanup queue. Preserve enough metadata to retry, make operations idempotent, and reconcile storage objects with database references.

Acceptance check. Inject failure at every storage/database boundary and verify recovery can reach a consistent final state without silently losing content.

APP-08 | Password-reset consumption is not atomic

Priority: High. Concurrency risk demonstrated by code.

Observed behavior. ResetPasswordAsync reads a row where UsedAt is null, then changes the password and marks UsedAt. Two independent requests can both finish the initial read before either saves.

Consequence. The single-use token rule is not protected against overlapping requests.

Recommended correction. Consume the token with a conditional update or lock/version check inside a transaction that also changes the password.

Acceptance check. Coordinate two test requests so they reach consumption concurrently. Assert exactly one succeeds and that password state agrees with that request.

APP-09 | Generic circle PATCH bypasses hierarchy invariants

Priority: High. Confirmed alternate-path defect.

Observed behavior. The generic PATCH applies ParentId and recalculates only the edited circle RootId. It does not use the dedicated cycle check or update descendant roots.

Consequence. Moving a subtree through generic PATCH can leave descendants assigned to the previous root.

Recommended correction. Route all hierarchy mutations through one service or prohibit ParentId changes in the generic endpoint.

Acceptance check. Move a three-level subtree between roots using every public mutation path. Compare all descendant roots and test self-parenting and descendant-parenting requests.

APP-10 | The dedicated hierarchy check uses an unlocked pre-transaction snapshot

Priority: High. Concurrency risk demonstrated by code.

Observed behavior. The dedicated path correctly checks a single request against its loaded graph, but reads and validation occur before the transaction starts. Two concurrent moves can validate against an older graph and each apply a conflicting update.

Consequence. For example, concurrent moves of A under B and B under A can both pass their own snapshot check.

Recommended correction. Include reads and validation in an appropriate locking/serializable strategy, or serialize hierarchy mutations with a shared database-level lock.

Acceptance check. Run opposing moves concurrently in a disposable database. Require that at most one commits, no parent cycle exists, and roots agree with ancestry.

APP-11 | Lifecycle edits and deletions leave the displayed phase stale

Priority: High. Confirmed synchronization gap.

Observed behavior. Creating a latest history entry updates the circle phase name. PATCH and DELETE save their history changes without recomputing that field.

Consequence. The history and the circle display can disagree after editing/removing the latest row or moving it to another circle.

Recommended correction. Centralize phase derivation across create, edit, delete, reassignment and phase rename. Decide latest-record versus effective-at-time semantics and a deterministic tie rule.

Acceptance check. Exercise all mutation paths, including deletion of the sole history row, same-date ties and a future row. Compare the stored/read model with the defined rule.

APP-12 | Permission cache invalidation reaches only one API process

Priority: Medium. Deployment-dependent risk.

Observed behavior. Permissions use IMemoryCache with a default 60-second duration. The update controller clears its local cache only.

Consequence. In a multi-instance deployment, a permission removal can remain ineffective on another instance until its cache expires.

Recommended correction. Use versioned grants, shared invalidation or a documented short revocation delay. Treat sensitive permission removals consistently across all instances rather than assuming a local cache clear is global.

Acceptance check. Warm the same role on two API processes, revoke a permission through one, and immediately check the other.

APP-13 | Open self-registration creates active accounts with directory access

Priority: High. Policy-dependent exposure.

Observed behavior. The signup endpoint creates an active User account and returns a token. The inspected path has no invite, verified-email or approval gate.

Consequence. If this application is intended to be restricted to staff, API reachability plus self-registration can admit an unapproved person to that directory and other default read capabilities.

Recommended correction. Have the owner define enrollment policy. For a staff-only deployment, use controlled provisioning/invitations and verified identity before activating access.

Acceptance check. Test enrollment from an unapproved synthetic identity and verify it cannot access organizational data until the required approval/verification succeeds.

APP-14 | Dashboard and profiles issue avoidable sequential queries

Priority: Medium. Confirmed query-shape issue.

Observed behavior. The dashboard executes eight sequential counts and loads the user ID/name set. A profile loads its circle IDs and then makes two queries per circle to obtain circle and lead-link data.

Consequence. Round trips increase with the number of circles per person and can dominate response time when the database is remote.

Recommended correction. Use projections and batched/set-based queries, fetch only relevant display names, and consider appropriate caching after measurement.

Acceptance check. Capture command counts and p50/p95 latency for profiles with 1, 10 and 50 circles in a representative test environment. Confirm the optimized output is equivalent.

APP-15 | Profile lead-link display selects an arbitrary matching person

Priority: Medium. Confirmed selection ambiguity.

Observed behavior. The profile takes FirstOrDefaultAsync over lead-link assignments without an ordering or a singular-cardinality constraint. That query does not filter inactive users or validity intervals.

Consequence. The displayed person can be incomplete or unstable when several holders qualify.

Recommended correction. Display all eligible lead links or enforce an explicit single-holder policy with deterministic selection. Define activity and effective-date filters consistently with organizational membership.

Acceptance check. Use a circle with two lead-link holders, an inactive holder and dated assignments. Verify the display follows the documented rule regardless of insertion order.

APP-16 | Frontend session lifetime can diverge from API-token lifetime

Priority: Medium. Configuration-dependent integration gap.

Observed behavior. The frontend stores tokenExpiresAt but does not use it in the inspected JWT/session callbacks. Its session maxAge is fixed at eight hours.

Consequence. If the backend lifetime is shortened, the UI can retain a session after API authorization expires.

Recommended correction. Align session validity with actual token expiry. On expiry, require reauthentication or implement an explicit secure refresh design; make API 401 handling end the invalid session coherently.

Acceptance check. Set a short backend token lifetime in a test environment. Verify that the visible session expires or reauthenticates at the correct time, including after an already issued token expires.

APP-17 | Single-holder role policy conflicts with the supplied import

Priority: High. Confirmed data/rule inconsistency.

Observed behavior. Assign allows multiple holders only when the definition name equals Mitglied. The database permits multiple distinct users per role.

Consequence. The imported dataset and API creation rule disagree.

Recommended correction. Decide which policy is correct before modifying data. Represent allowed cardinality explicitly rather than deriving it from display text.

Acceptance check. Rebuild representative imported multi-holder roles through the API after the agreed fix. Test a definition rename and concurrent assignments; both must preserve the intended holder policy.

APP-18 | Client-supplied creator IDs can survive entity-bound create requests

Priority: Medium. Confirmed trust-boundary defect.

Observed behavior. Several create endpoints bind directly to auditable entity types. For added rows, WendeAuditFelderAn sets CreatedById only when the supplied value is empty.

Consequence. The entity creator metadata can misattribute who created a record.

Recommended correction. Use request DTOs that exclude server-owned audit fields and set CreatedById from trusted identity for ordinary requests.

Acceptance check. Create a test circle with a fabricated CreatedById using an otherwise authorized account. The stored creator must be the authenticated account, and restricted imports must follow their own explicit rules.

APP-19 | UTC date writes and log filters fail at the PostgreSQL boundary

Priority: High. Reproduced date-handling defect.

Observed behavior. Valid ISO date-time strings ending in Z produce HTTP 500 on five tested date fields. The exception states that DateTime Kind=Unspecified cannot be written to timestamp with time zone.

Consequence. Meeting scheduling, KPI measurement creation and due/review-date changes can fail after the user has completed an otherwise valid form.

Recommended correction. Choose an explicit temporal contract: UTC instants for timestamps and date-only values for business dates.

Acceptance check. Round-trip all five affected fields and log date filters through the browser and API with UTC, explicit offsets, cleared optional values and daylight-saving boundaries. Read back the intended instant/date on the deployment PostgreSQL version.

APP-20 | Updating an individual mailing list fails during membership replacement

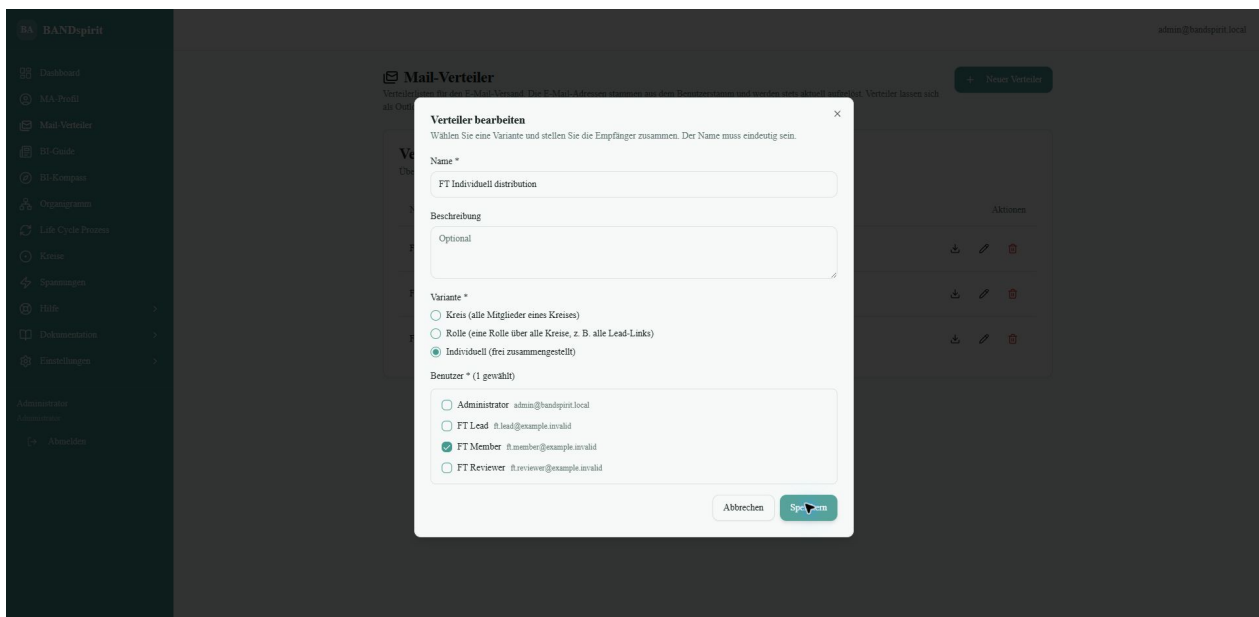
Priority: High. Reproduced update failure.

Observed behavior. Creating an individual list and resolving two recipients work. Updating that same list returns HTTP 500 with DbUpdateConcurrencyException; the requested values do not persist.

Consequence. Users can create a list that they cannot subsequently maintain.

Recommended correction. Inspect EF entity states and generated key handling during replacement. Prefer a set difference: retain unchanged members, explicitly add new rows and remove only departed members inside a transaction.

Acceptance check. Create a two-member list, rename it, remove one member, add another, save the unchanged set and delete it. Verify both membership rows and recipient resolution after every save.



Screenshot evidence. WEB-010. Individual-list edit remains open after the rejected update. HTTP 500 and persistence checks establish the failure; the image alone does not show the backend exception.

APP-21 | Single-role expansion loses assignments that collection expansion returns

Priority: Medium. Reproduced query-shape defect.

Observed behavior. The single-role GET with \$expand=assignments returns an empty collection, while filtering the role collection to the same ID with the same expansion returns both assignments. PostgreSQL confirms the two rows.

Consequence. Different screens can report different holders for the same role.

Recommended correction. Keep the single-entity query composable using the appropriate OData SingleResult/query return shape, or explicitly project a detail DTO with the required assignments.

Acceptance check. With zero, one and two assignments, compare single GET, filtered collection GET and direct database counts.

APP-22 | User administration accepts an invalid email address

Priority: Medium. Reproduced input-validation gap.

Observed behavior. The user-administration POST accepts email value bad and returns HTTP 201. Its UserDto exposes unconstrained Name, Email and Role strings; model annotations on a different entity do not automatically validate this input DTO.

Consequence. Direct API callers can create values the UI treats as invalid.

Recommended correction. Define separate validated create/update/read contracts. Apply required, length, email and role-catalog validation at the API boundary; normalize consistently and keep relevant database constraints.

Acceptance check. Reject malformed/blank/oversized values through direct API requests with field-specific 400 responses; round-trip valid names and addresses.

3 Frontend, visual design and interaction findings

These 32 findings combine browser observations with source evidence. The current pass uses known fixtures persisted through the real backend, with explicitly documented SQL fixture exceptions. Earlier synthetic-API observations retain their historical scope. An acceptance check describes the required fix verification, not a test automatically claimed as passed. Screenshot captions identify the scenario and what the image proves.

UI-01 | Help tabs retain stale state after sidebar navigation

Priority: Medium. Reproduced navigation defect.

Observed behavior. Open Hilfe > Suche and then choose FAQ in the sidebar without reloading. The URL changes to /hilfe?

Consequence. The address, sidebar selection and visible content disagree.

Recommended correction. Use the URL as the tab state, with a validated default, and update the query string when a tab is selected.

Acceptance check. Navigate Suche -> FAQ -> Tickets through both navigation surfaces, then use Back, Forward and reload. The URL and selected panel must agree at each step, including an empty FAQ collection.

UI-02 | Sidebar selection ignores tab query parameters

Priority: Low. Reproduced navigation defect.

Observed behavior. Active-link comparisons use pathname against href strings that sometimes contain query parameters. On /hilfe?

Consequence. Users lose a reliable indication of their current location.

Recommended correction. Represent destinations as a pathname plus relevant query keys, and share the matching function with tab navigation.

Acceptance check. For every sibling link sharing a pathname, assert exactly one appropriate active item at direct entry and after client navigation.

UI-03 | The organigram extends beyond a phone-width viewport

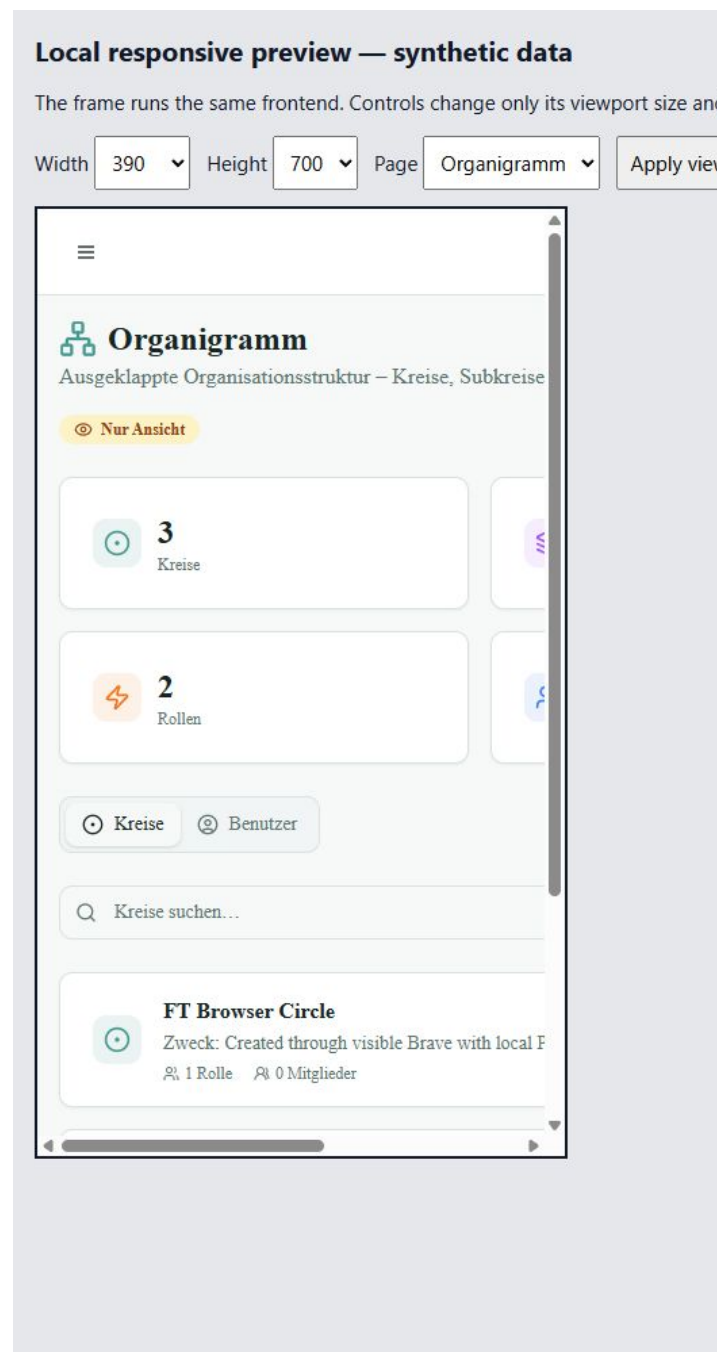
Priority: Medium. Reproduced responsive layout defect.

Observed behavior. The real frontend was rendered in a same-origin iframe with a 390 by 700 CSS-pixel viewport and seven synthetic nested circles. A horizontal scrollbar appears and right-side summary content, account text and circle actions extend outside the visible area.

Consequence. A narrow-screen user must pan sideways to discover information and controls.

Recommended correction. Allow flex content to shrink with min-width:0, keep the desktop sidebar width predictable, stack card actions at small breakpoints and bound visual indentation.

Acceptance check. At 320, 390, 768 and 1280 CSS pixels, test long names, expanded branches and at least six levels. All ordinary page controls must remain reachable without page-level horizontal panning; any intentional two-dimensional graph region must be clear and independently usable.



Screenshot evidence. WEB-049. 390 by 700 CSS-pixel organigram viewport. The page exceeds the frame width; the right side of the original capture is clipped here only to enlarge the phone-sized evidence.

UI-04 | The mobile sidebar lacks a complete drawer interaction model

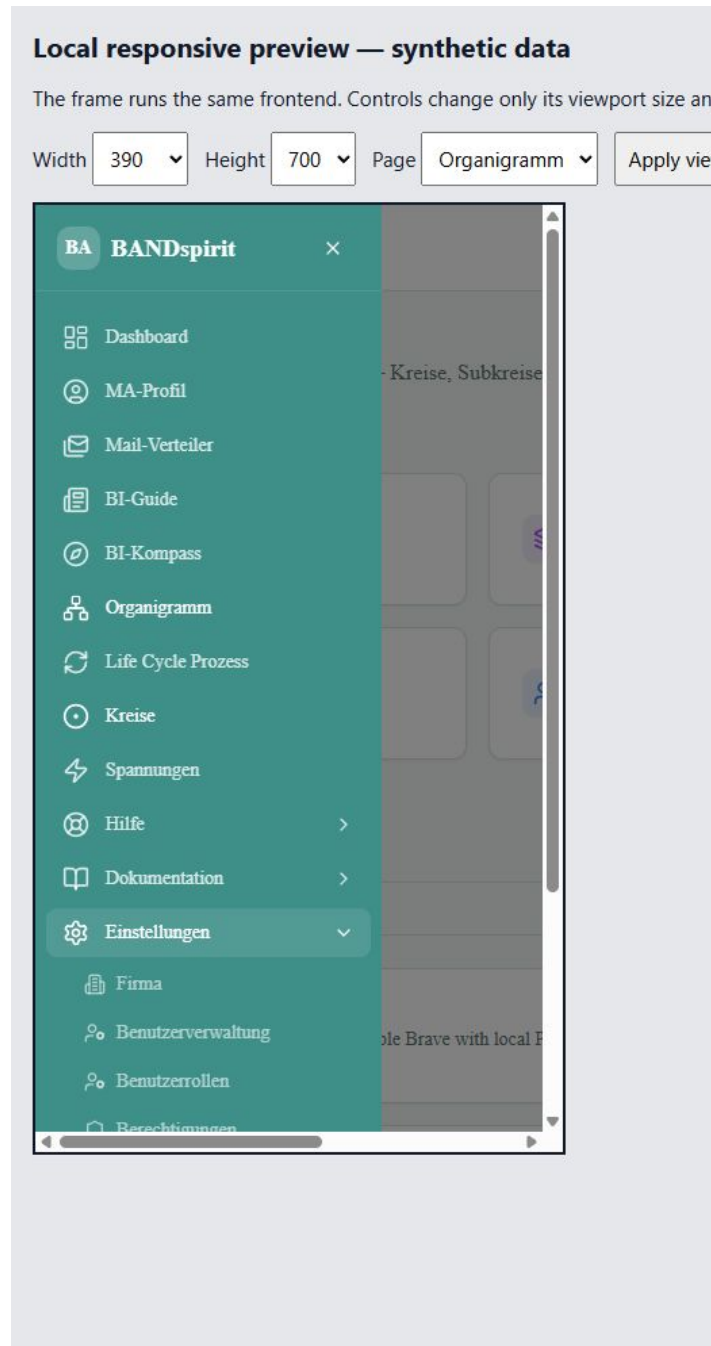
Priority: Medium. Reproduced keyboard interaction defect.

Observed behavior. The sidebar is moved off screen with a transform. The markup does not provide a matching inert/hidden state for its controls, a focus-management cycle, Escape handling or expanded/control attributes on the toggle.

Consequence. At constrained heights or with keyboard navigation, focus may reach visually hidden controls, the menu may be difficult to dismiss, or later navigation items may be hard to reach.

Recommended correction. Use a tested accessible drawer primitive or implement opening focus, background isolation, Escape, focus restoration and close-on-navigation.

Acceptance check. With a keyboard at narrow width and short height, open and close the menu, expand all groups, reach the final item and navigate away. Focus must remain visible, off-screen items must not be tabbable while closed, and dismissal must return focus predictably.



Screenshot evidence. WEB-051. Mobile menu remains open after Escape. Keyboard location and hidden-focus coordinates are recorded in WEB-050; a screenshot cannot by itself prove keyboard focus.

UI-05 | Sidebar text contrast is too low for ordinary navigation labels

Priority: Medium. Measured accessibility defect.

Observed behavior. The measured sidebar background is RGB(62,143,136). Full white on that color gives approximately 3.83:1 contrast; white at 80 percent opacity gives 3.04:1 and at 60 percent gives 2.37:1 after alpha compositing.

Consequence. The low-opacity submenu and secondary account text are especially difficult to read.

Recommended correction. Choose foreground/background pairs that meet the contrast requirement for actual sizes and weights. Darken the teal background or choose a suitably dark foreground, then verify hover, selected and disabled states separately.

Acceptance check. Calculate contrast from the final rendered colors, including opacity and overlays. Ordinary active navigation text must meet 4.5:1, with the appropriate separate criteria for large text and non-text controls.

UI-06 | Many pages send notifications to an unmounted toast system

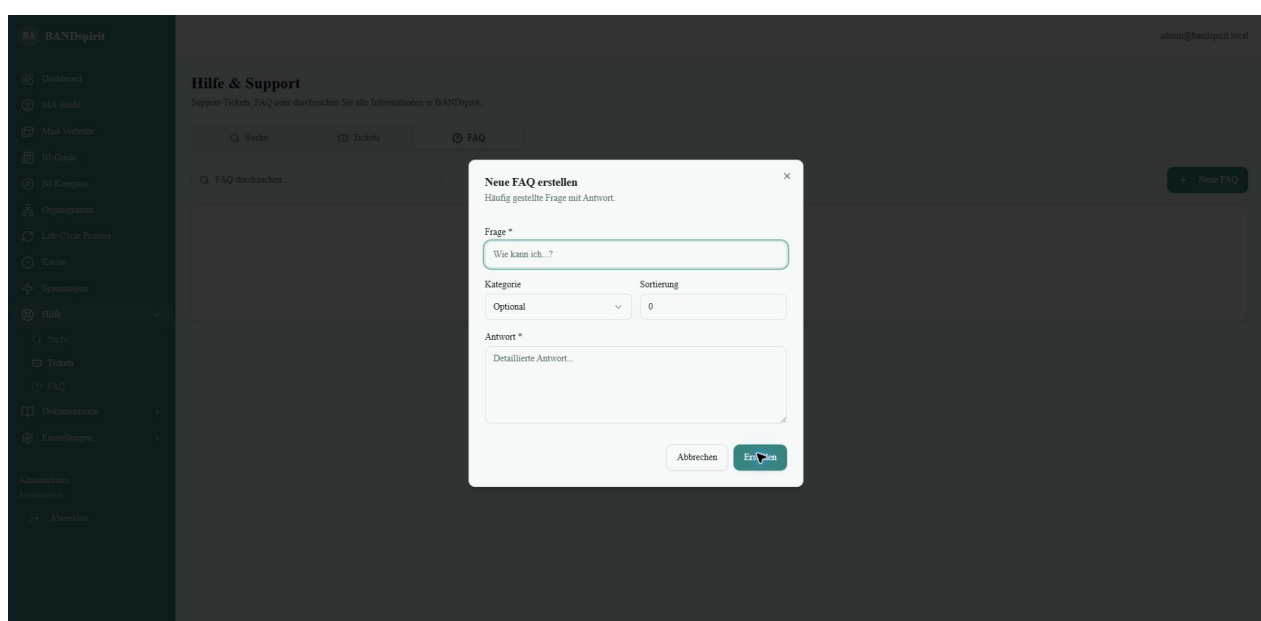
Priority: High. Reproduced feedback defect.

Observed behavior. The root mounts the Sonner renderer, while Help and several settings/profile pages call the separate hooks/use-toast store. Its matching Toaster is not mounted in the inspected application tree.

Consequence. Users cannot reliably distinguish validation failure, server failure and a successful save.

Recommended correction. Standardize notification calls and their renderer, or deliberately mount both with distinct purposes. Put required-field errors next to their fields, associate them programmatically, and focus the first invalid field.

Acceptance check. Trigger blank input, simulated server validation, network failure and success on each affected form. The appropriate message must appear once, be exposed to assistive technology and explain the next action.



Screenshot evidence. WEB-002. Empty FAQ submission leaves the dialog without a useful required-field

message.

UI-07 | Unavailable data is presented as zero counts or no search results

Priority: High. Reproduced failure-state defect.

Observed behavior. When the local fixture returns HTTP 503, the dashboard finishes loading and displays zero counts without an error/retry state. A Help search during the same simulated failure says that no results were found and suggests another search term.

Consequence. Zero and no matches are valid business results, so users can mistake unavailable information for an empty organization or an unsuccessful search.

Recommended correction. Model loading, successful empty, successful populated, failed and stale states explicitly. Keep the query and any last good data, visibly identify failed refreshes and provide retry.

Acceptance check. Exercise 200 with data, 200 empty, 401/403, 503 and a disconnected request. Each state must have appropriate text and actions.

UI-08 | Several filters and search placeholders promise behavior not implemented

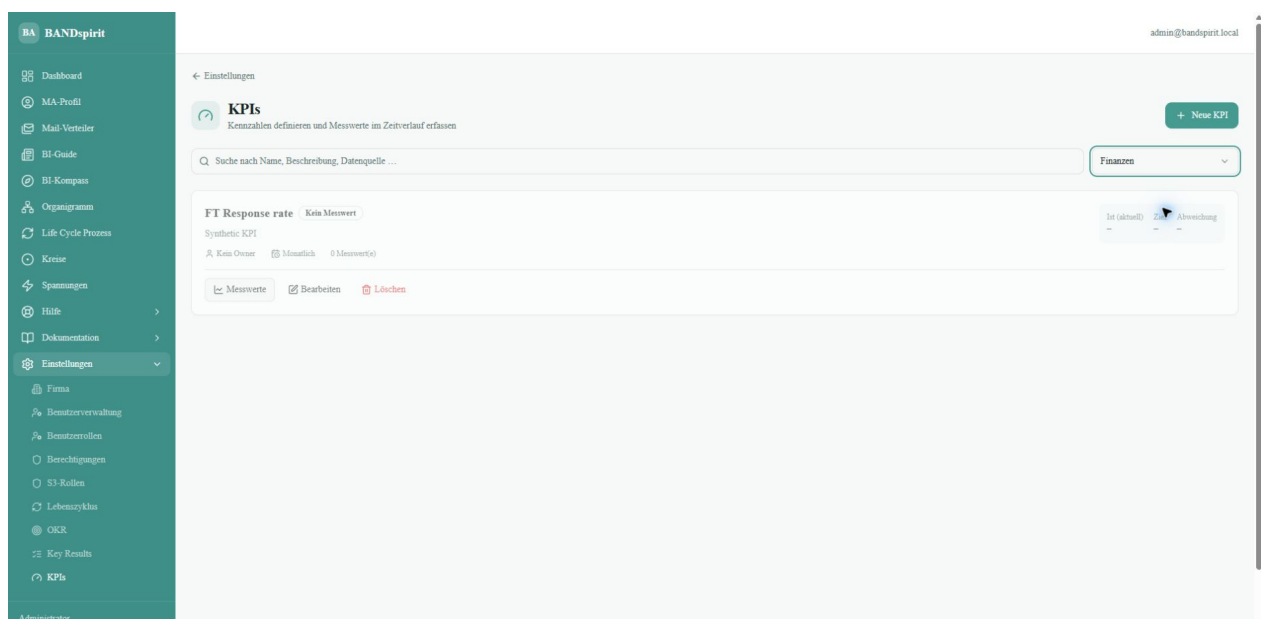
Priority: Medium. Reproduced incomplete filter behavior.

Observed behavior. Selecting Finanzen in the KPI category control leaves the uncategorized synthetic row visible and issues the same collection query; the loader explicitly omits category filtering. The Key Result status selection is likewise not applied to a corresponding backend field.

Consequence. Controls imply that the displayed records have been narrowed when they have not.

Recommended correction. Remove or disable unsupported controls with an explanation until their domain fields exist, or implement the full supported query.

Acceptance check. Against the real API, seed records that differ only in each advertised field and test every option independently and in combination. Check the outgoing query, returned records and visible result count; changing sort order must not be confused with filtering.



Screenshot evidence. WEB-046. Finanzen is selected while the unrelated KPI remains visible. The omitted filter in the loader provides the matching source evidence.

UI-09 | Global search describes modules it never searches

Priority: Medium. Source-confirmed scope mismatch.

Observed behavior. The search description advertises all information and names clients, contacts, intakes, reports, conversations and assignments. The implementation requests only BiGuideNews and SupportTickets and searches their titles.

Consequence. A user can interpret a negative result as evidence that a person or business record does not exist.

Recommended correction. Describe the current scope precisely, including the fields searched, or introduce a permission-aware search endpoint that covers the advertised resources.

Acceptance check. Create a uniquely identifiable record in every advertised searchable resource and field. Verify discovery for an authorized user and non-disclosure for an unauthorized user.

UI-10 | Search results lead to whole lists instead of the matched item

Priority: Low. Source-confirmed navigation weakness.

Observed behavior. Every news result receives /organisation/bi-guide and every support result receives /hilfe? tab=tickets.

Consequence. Search saves less work than its result presentation implies.

Recommended correction. Provide a stable per-record route, or a validated selected-record query parameter that opens and focuses the corresponding detail.

Acceptance check. Find two records with similar titles and open each result. The correct record must be displayed or highlighted immediately, with Back returning to the original search state.

UI-11 | Support ticket rendering and editing assume undeclared backend fields

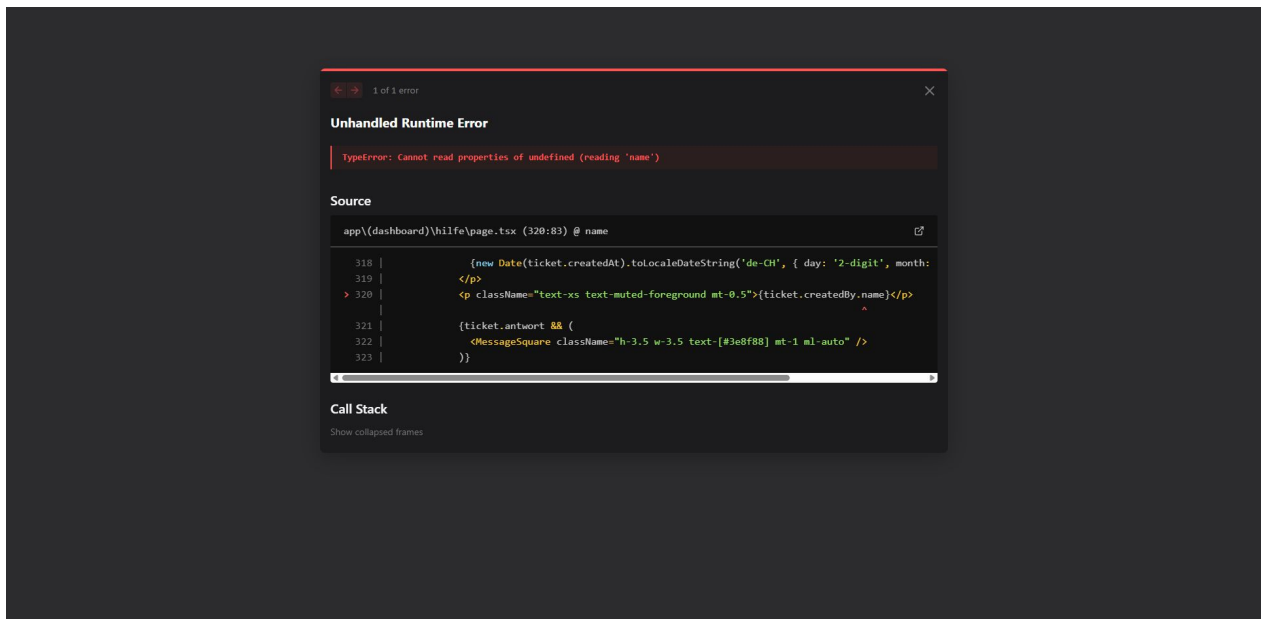
Priority: High. Reproduced populated-page crash and contract mismatch.

Observed behavior. The frontend expects kategorie, antwort, createdBy and assignedTo, renders ticket.createdBy.name without a fallback, and sends category/response fields on writes. The inspected model instead has Titel, Beschreibung, Status, Prioritaet and ErstellerId plus identity/audit fields; it does not declare that response/category/assignee shape.

Consequence. A real response shaped like the declared model can break author rendering or leave form input without a persistence destination.

Recommended correction. Define explicit ticket list/detail/write DTOs, including author and response semantics, and generate or validate TypeScript types from the contract.

Acceptance check. Create a ticket through the real API, reload it, display its author, edit status and response, and verify persistence and authorization.



Screenshot evidence. WEB-006. A populated support-ticket page raises a client exception at the assumed nested author. This is the local development error overlay, not a claim about production error-page styling.

UI-12 | The tension form posts English field names to a German model

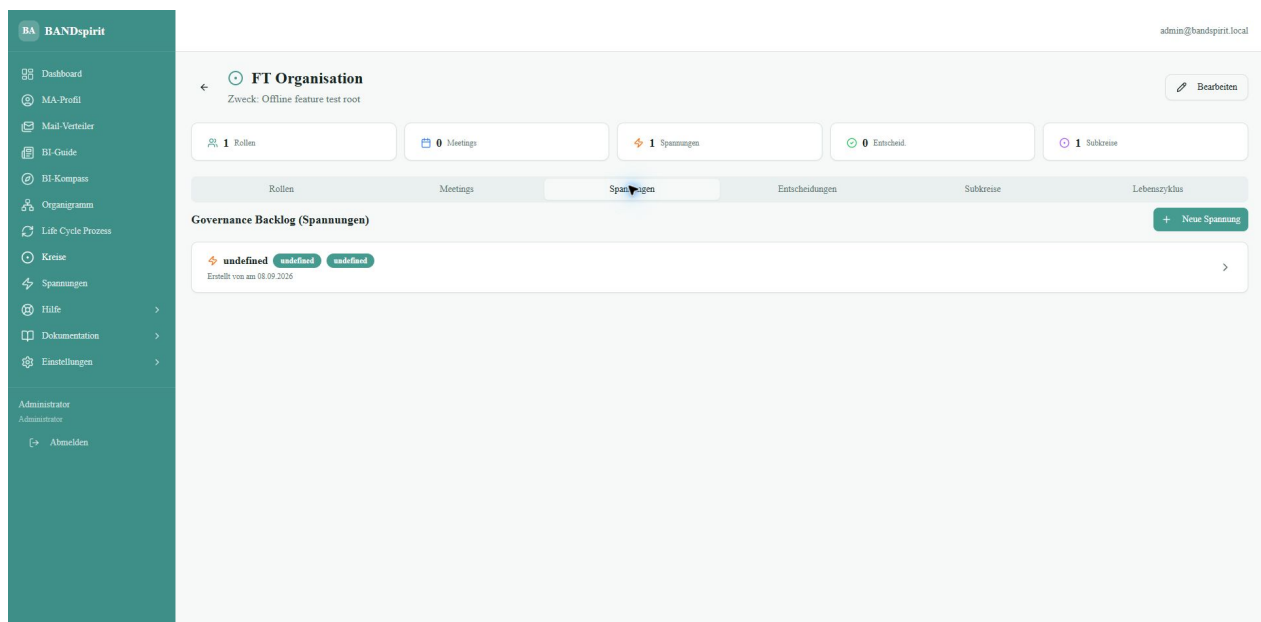
Priority: High. Reproduced frontend/API contract mismatch.

Observed behavior. driverForm contains title, description, priority and circleId and is posted directly to /odata/Drivers. The inspected S3Driver model uses Titel, Beschreibung, Entscheid, EntscheidDatum and CircleId, and does not define the displayed priority field.

Consequence. A form that looks complete does not match the declared write contract.

Recommended correction. Introduce a typed write adapter with the actual API names and decide whether priority belongs in the domain model.

Acceptance check. Submit a unique title and description, select a circle and exercise each supported classification. Read the record back from the real API and compare every visible field.



Screenshot evidence. WEB-025. A persisted driver renders undefined labels in the tension list. WEB-023 and WEB-026 separately record failed creation and detail navigation.

UI-13 | The OKR form collects owner, team and period without matching persistence fields

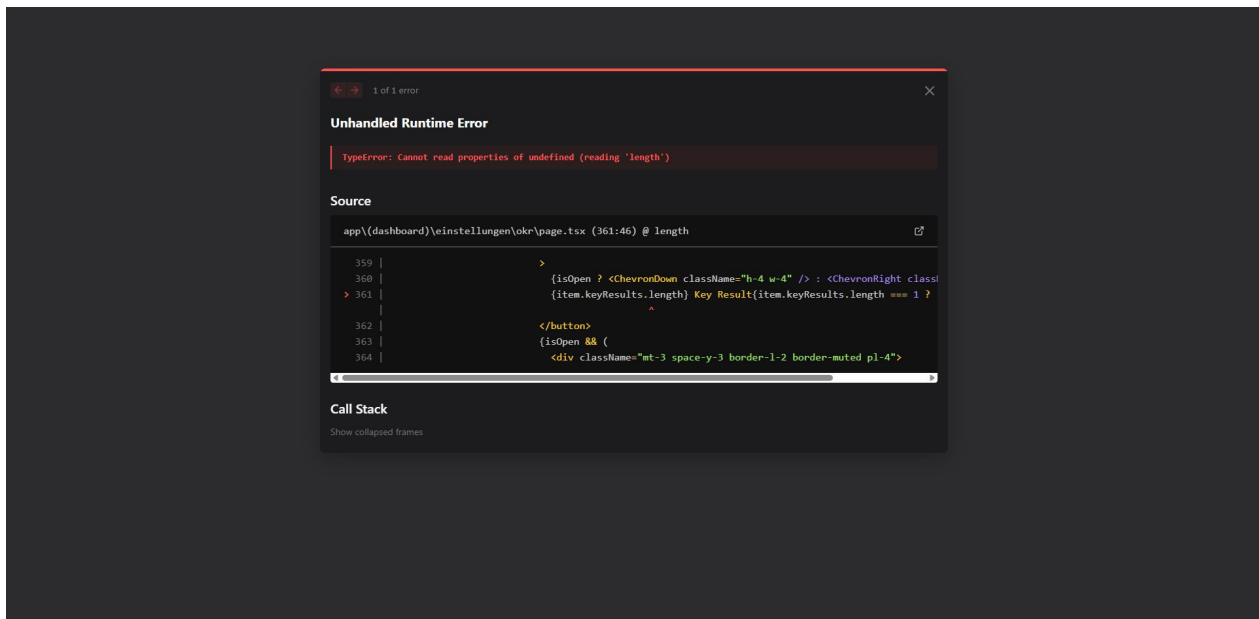
Priority: High. Reproduced populated-page crash and contract mismatch.

Observed behavior. The form state includes ownerId, team and zeitraum and is submitted as a whole object. The inspected OKR model does not define those properties; it uses Faelligkeit for a due date rather than the displayed period concept.

Consequence. Users may invest effort in information that has no defined storage contract.

Recommended correction. Agree on objective ownership and time-period semantics, add explicit fields/relationships and migrations if required, and expose a dedicated DTO.

Acceptance check. Round-trip every editable field against the real backend, including clearing optional values and changing the period. Confirm that reload, filtering and exported views use the same persisted meaning.



Screenshot evidence. WEB-040. Persisted OKR data triggers the missing keyResults collection exception. The overlay identifies the failing client expression.

UI-14 | Key Result ownership and numeric values use divergent names

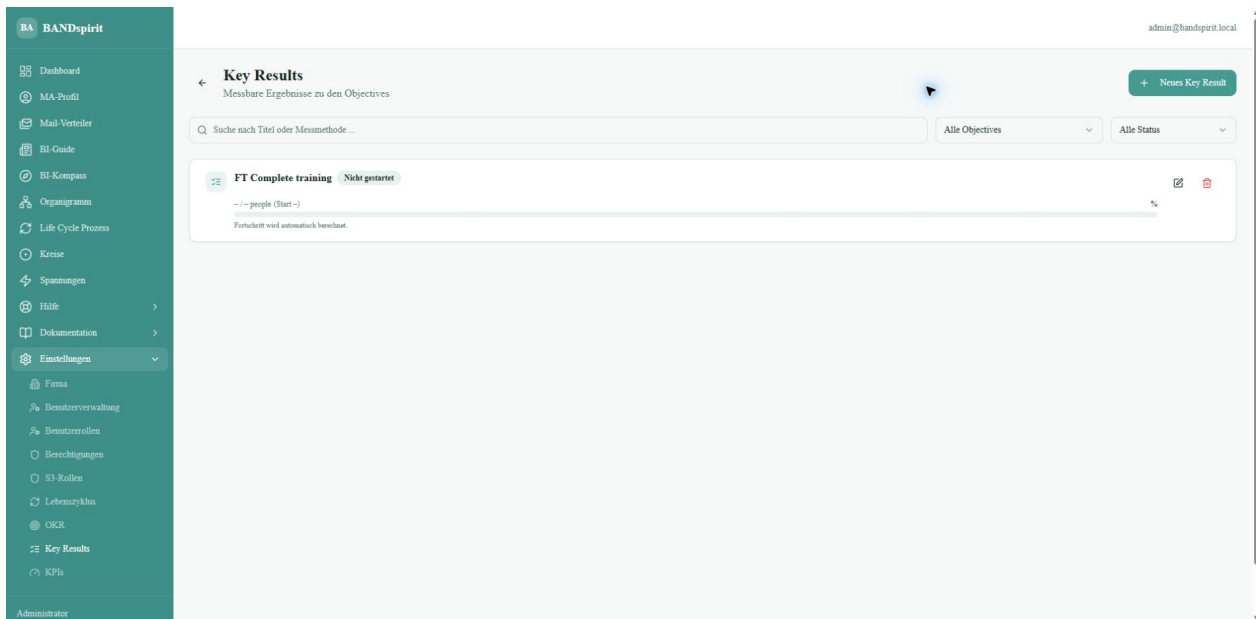
Priority: High. Reproduced value mapping and save defect.

Observed behavior. The UI uses `objectiveId` while the backend relationship is `OkrId`. It uses `startwert`, `zielwert` and `aktuellerWert` where the declared model produces `startWert`, `zielWert` and `istWert` under lower-camel naming.

Consequence. Relationships or values can fail to bind or display as missing after a server response.

Recommended correction. Map a documented Key Result DTO explicitly, normalize numeric input, and use consistent null and unit rules.

Acceptance check. Create an objective and a Key Result, then read back its `OkrId` and its start, target and current values from the real API. Test zero, decimals, cleared input and invalid numbers.



Screenshot evidence. WEB-041. Key-result values exist in PostgreSQL but display as missing values; WEB-042 records the failed edit form.

UI-15 | KPI editing assumes a richer model than the API declares

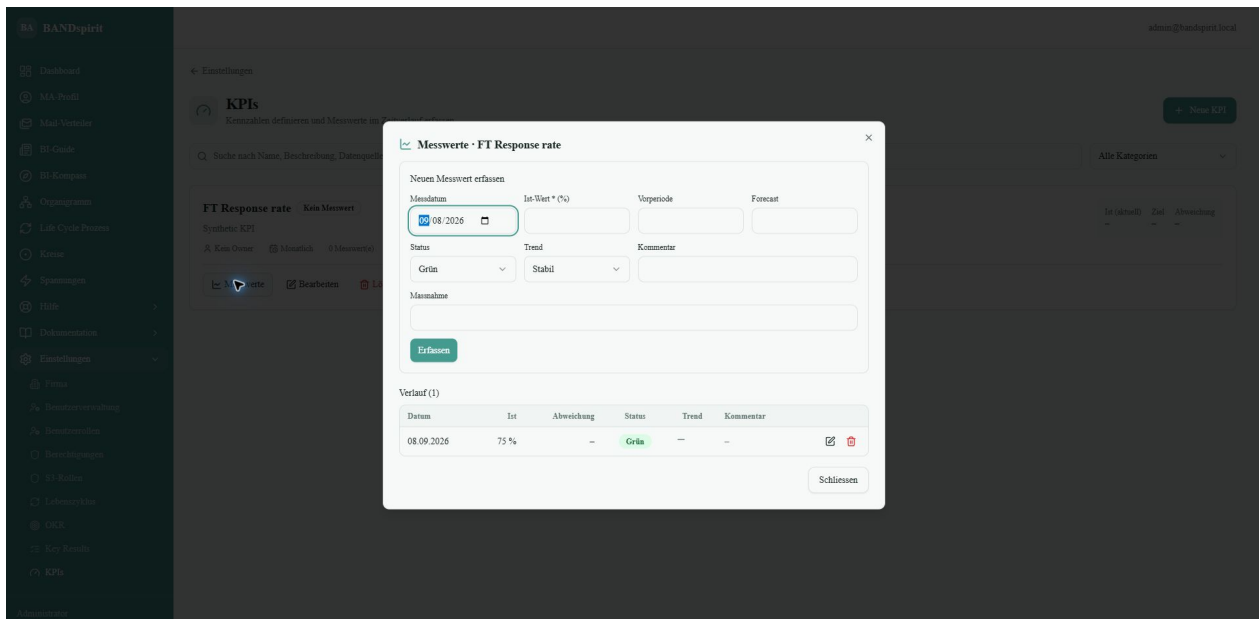
Priority: High. Reproduced KPI presentation and write defects.

Observed behavior. The definition editor collects category, owner, Key Result linkage, target/baseline, formula, data source and validity information beyond the inspected KpiDefinition fields. That model declares Name, Beschreibung, Einheit, Warnschwelle, KritischeSchwelle and Messintervall with its measurement relationship.

Consequence. The frontend presents a measurement-management workflow whose persistence and validation semantics are incomplete.

Recommended correction. Design definition and measurement DTOs together with the domain rules, then implement the selected fields end to end.

Acceptance check. Against the real backend, save, reload and update every supported field on a definition and a measurement. Verify units, threshold boundaries, time zones, optional values and authorization.



Screenshot evidence. WEB-043. KPI history displays the local measurement, while the card summary still reports no value/count. The history fixture was inserted with SQL after API creation failed.

UI-16 | Frontend lists do not follow server-driven OData pagination

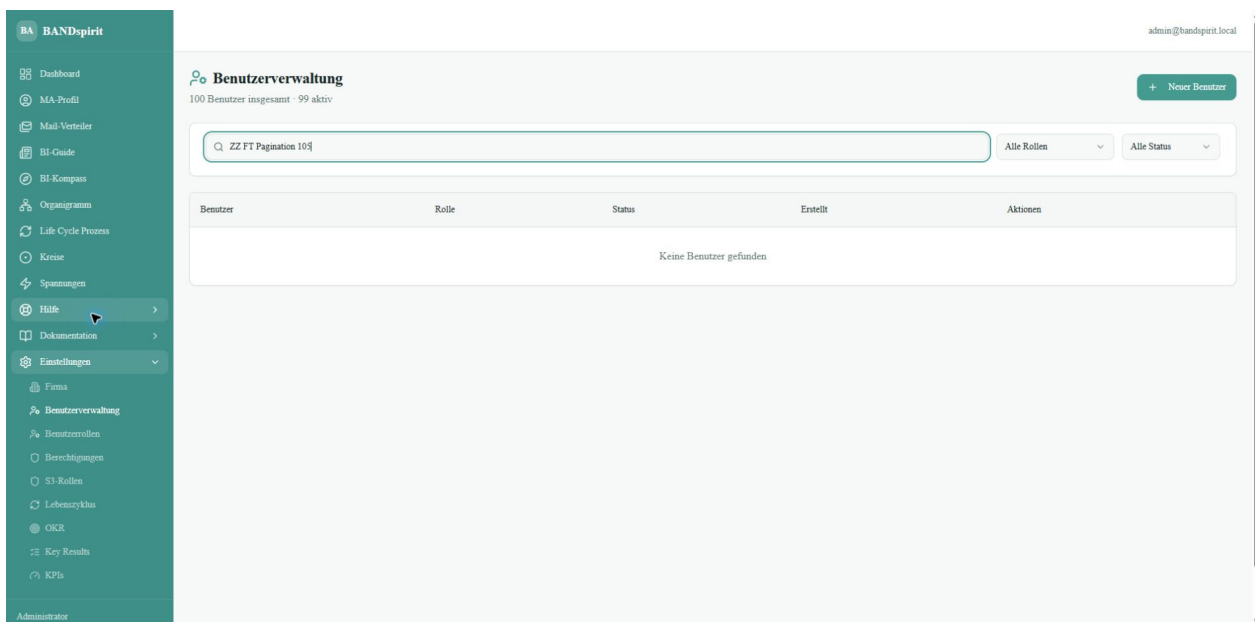
Priority: Medium. Reproduced incomplete user list.

Observed behavior. The user endpoint applies a server page size of 100, while the frontend takes one response.value and filters that array locally. The reviewed response type and collection logic do not handle @odata.nextLink.

Consequence. Later users may be absent from tables, client-side searches and selectors.

Recommended correction. Add server paging and show the current page and total. For small selector datasets, follow every nextLink and show progress or a failure message.

Acceptance check. Seed at least 101 users in a real integration environment and find/select the final record. Check paging, search and count accuracy with 308 or more records, including a failure on a later page.



Screenshot evidence. WEB-032. The UI searches only its first 100 users; the later-page Pagination 105 record is absent from the result. API paging evidence establishes the full collection size.

UI-17 | Circle summaries substitute zeros for data they have not loaded

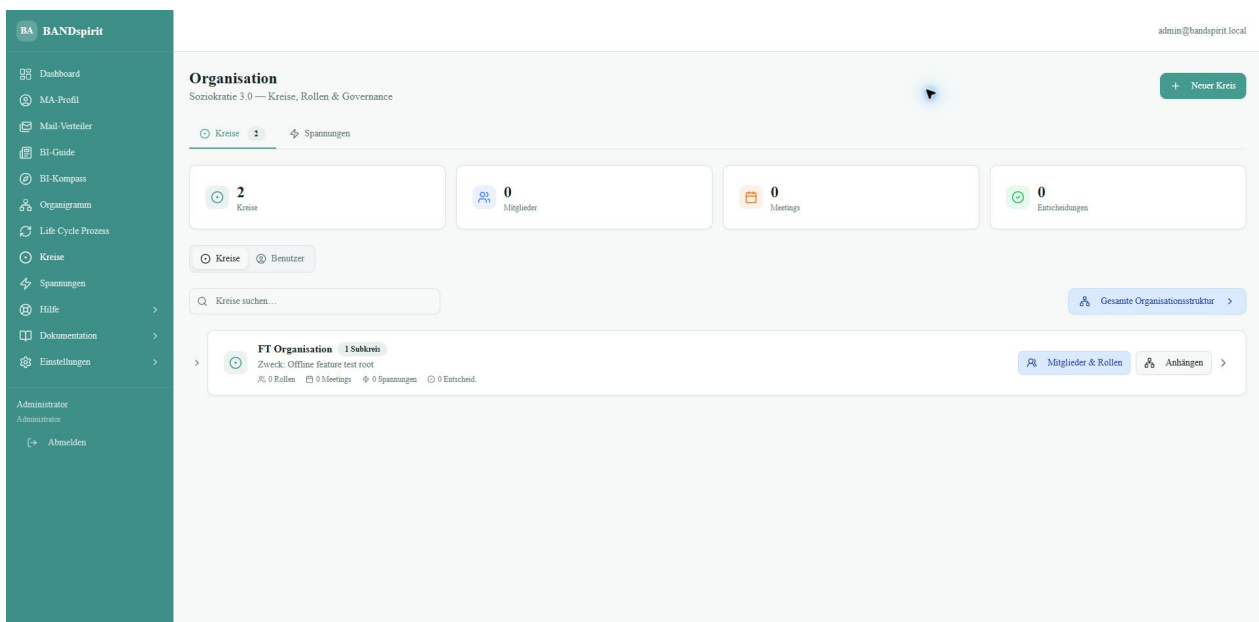
Priority: Medium. Reproduced contradictory data presentation.

Observed behavior. The collection adapter assigns roles:[] and sets role, meeting, driver and decision counts to zero, with a comment that roles load only in the detail page. In the browser, a circle with a synthetic role is nevertheless labeled 0 Rollen, and the membership summary is zero.

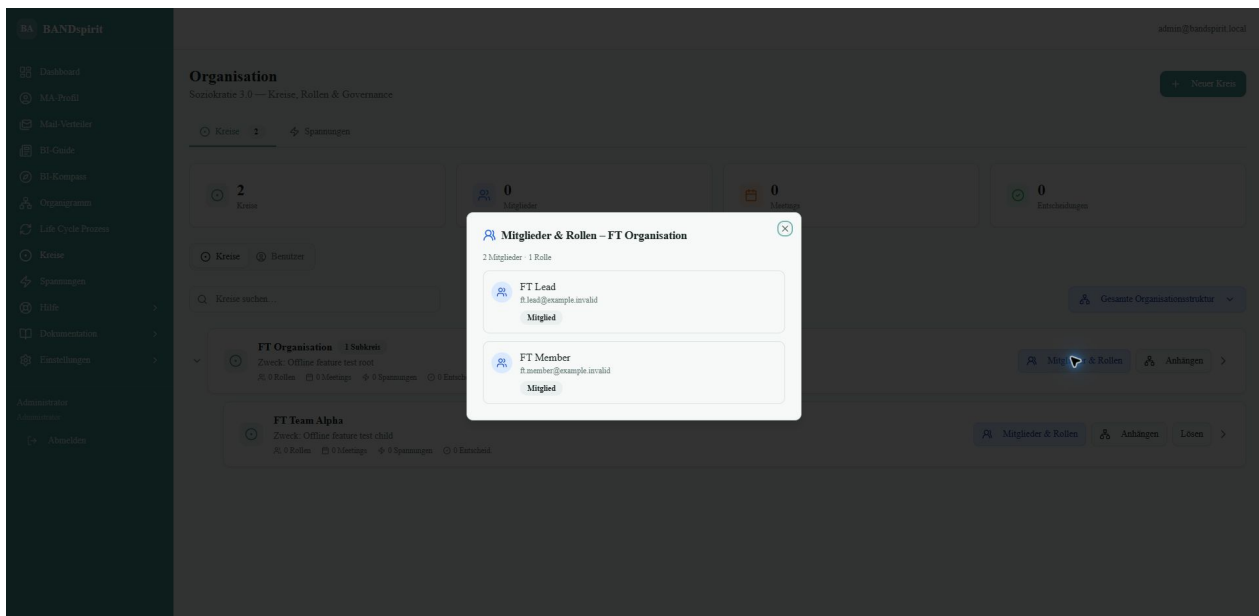
Consequence. A fabricated zero is indistinguishable from a verified empty collection.

Recommended correction. Fetch authorized summary aggregates from the API or show an explicit unavailable/not-loaded marker. Do not infer member counts from missing role arrays.

Acceptance check. Use circles with known nonzero counts and one genuinely empty circle. Compare list, graph and detail displays against authoritative aggregates.



Screenshot evidence. WEB-014. Circle overview substitutes zero summaries for known data.



Screenshot evidence. WEB-015. The same local circle exposes its actual members and roles in the dialog, contradicting the overview summary.

UI-18 | Application Log is hidden from the managed administrator navigation

Priority: Medium. Reproduced navigation omission.

Observed behavior. The Application-Log navigation entry has an applog:read permission but no accessKey. For managed roles, canSeeItem immediately rejects an item without that key before the role access map is evaluated.

Consequence. An administrative feature becomes undiscoverable through the main navigation.

Recommended correction. Give the entry a supported access key and align its role mapping with the effective permission model, or consolidate navigation visibility around one permission source.

Acceptance check. For Admin, member, lead and a custom role with explicit grants, compare expected menu items with accessible routes.

UI-19 | Application Log cannot filter the audit module the backend actually writes

Priority: Medium. Reproduced filter vocabulary mismatch.

Observed behavior. The module selector offers LOGIN, KLIENT, KONTAKT and BENUTZER. The inspected audit writer uses Datenaenderung.

Consequence. The log interface is poorly aligned with its own audit source and makes targeted investigation harder.

Recommended correction. Use a shared supported module catalog or an authorized distinct-values endpoint, with stable stored keys and localized labels.

Acceptance check. Create real audit events through supported operations and verify that each emitted module can be selected and returns the expected records.

The screenshot shows the 'Application-Log' interface. On the left is a sidebar with navigation items. The main area has a search bar containing 'S3LebenszyklusPhase'. A 'Modul' dropdown menu is open, displaying a list of modules: 'Alle Module', 'Login', 'Klient', 'Kontakt', and 'Benutzer'. Below the search bar is a table titled 'Log-Einträge'. The table has columns for 'Zeitpunkt', 'Modul', 'Aktion', 'Benutzer', and 'Objekt'. The visible rows show multiple entries for 'Datenaenderung' performed by 'Administrator' on 'S3LebenszyklusPhase'. The table indicates there are 10 entries in total.

Screenshot evidence. WEB-061. The visible audit module Datenaenderung is missing from the module selector, although searching the entity finds its records.

UI-20 | Icon actions and several form fields lack programmatic names

Priority: Medium. DOM- and source-confirmed accessibility defect.

Observed behavior. The FAQ edit/delete icon buttons appear as unnamed buttons in the browser accessibility snapshot. Several visible labels are not associated with an input by htmlFor/id, including the numeric Sortierung field.

Consequence. Keyboard and assistive-technology users cannot reliably identify an icon action or the purpose of a numeric/date field.

Recommended correction. Give icon-only buttons concise action names, use associated labels and stable IDs, and connect help/error text with aria-describedby.

Acceptance check. Inspect the accessibility tree and navigate the affected forms without a mouse. Each interactive control must expose its purpose and state; validation must identify the field and correction.

UI-21 | FAQ cards nest edit and delete buttons inside another button

Priority: Medium. DOM-confirmed invalid interaction structure.

Observed behavior. An FAQ card uses a button as its expand/collapse trigger and places the edit and delete Button components inside that trigger. With one FAQ entry, the live DOM contains two nested button relationships. stopPropagation in an event handler does not make nested interactive markup valid or define correct keyboard behavior.

Consequence. Focus and activation semantics are ambiguous, and browser parsing or hydration behavior can differ.

Recommended correction. Use a non-interactive card container with a dedicated accordion trigger and separate sibling action buttons.

Acceptance check. Confirm there are no nested interactive controls in the revised cards.

UI-22 | Help dialogs have no explicit vertical overflow strategy

Priority: Medium. Source-confirmed constrained-height risk.

Observed behavior. The shared dialog content is fixed at the viewport center with a half-height translation. The ticket and FAQ callers add a maximum width but no maximum height or overflow scrolling.

Consequence. The title, close control or submit action can become difficult to reach when the content is taller than the available viewport.

Recommended correction. Establish a shared dialog height/scroll policy using the dynamic viewport height, with appropriate safe-area spacing.

Acceptance check. Test the longest form and server-error state at a 560-pixel viewport height, increased text size and browser zoom, including an on-screen keyboard. Reach the first field, final action and close control without losing context.

UI-23 | The dashboard gives limited direction when activity sections are empty

Priority: Low. Observed design improvement.

Observed behavior. The baseline dashboard shows four summary cards and substantial unused space when news and review sections are empty. Cards have hover styling but no link/button behavior.

Consequence. A new or infrequent user receives little guidance on the next useful action and may try clicking a visually emphasized summary.

Recommended correction. Make useful summaries permission-aware drill-down links where a corresponding list exists, or remove interaction-like styling.

Acceptance check. With representative users, test whether they can explain each metric and find the next task without guidance. Cover empty, populated and failed states, and ensure any new card

links are keyboard accessible and land on a correctly filtered list.

UI-24 | The archive development proxy intercepts NextAuth routes

Priority: Medium. Reproduced development configuration defect.

Observed behavior. With the archive rewrite, `/api/auth/providers` and `/api/auth/session` were forwarded to the local backend fixture instead of reaching the NextAuth catch-all route, producing fixture 404 responses and preventing the expected login flow. Excluding the auth subtree from that rewrite in the isolated test copy restored the frontend session flow.

Consequence. A developer can misdiagnose a routing failure as bad credentials or a broken authentication service.

Recommended correction. Assign unambiguous route ownership: preserve the NextAuth subtree and route only the intended backend paths, or use a separate explicit backend prefix.

Acceptance check. On a clean checkout, request the provider/session endpoints and log in through the UI. Verify they reach NextAuth while an ordinary backend API call reaches the backend.

UI-25 | Frontend action visibility uses incompatible permission models

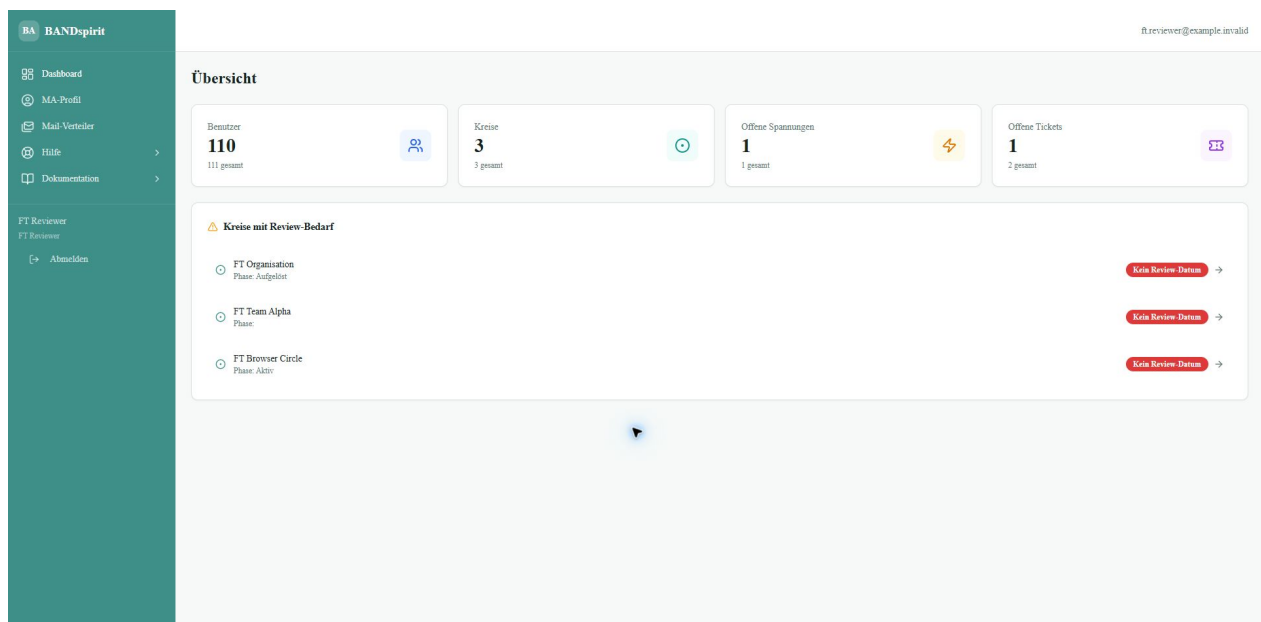
Priority: Medium. Reproduced permission visibility inconsistency.

Observed behavior. The shell recognizes a managed-role access map and role-name normalization, while static `hasPermission` logic describes a smaller fixed role set. Other pages use dynamically loaded permissions.

Consequence. Users can reach a page without the expected actions, or be presented with actions that the API rejects.

Recommended correction. Expose one effective capability contract to the frontend and use it consistently for navigation and actions, with explicit loading and failure states.

Acceptance check. Test Admin, ordinary member, Lead-Link and a custom role with explicit grants against the real permission endpoint. Compare navigation, buttons, direct routes and API decisions after granting and revoking a capability, including a refresh and expired session.



Screenshot evidence. WEB-068. The custom FT Reviewer sees Mail-Verteiler without a corresponding stored grant. API enforcement is a separate check; navigation visibility is not proof of granted access.

UI-26 | BI Compass crashes when rendering a persisted active version

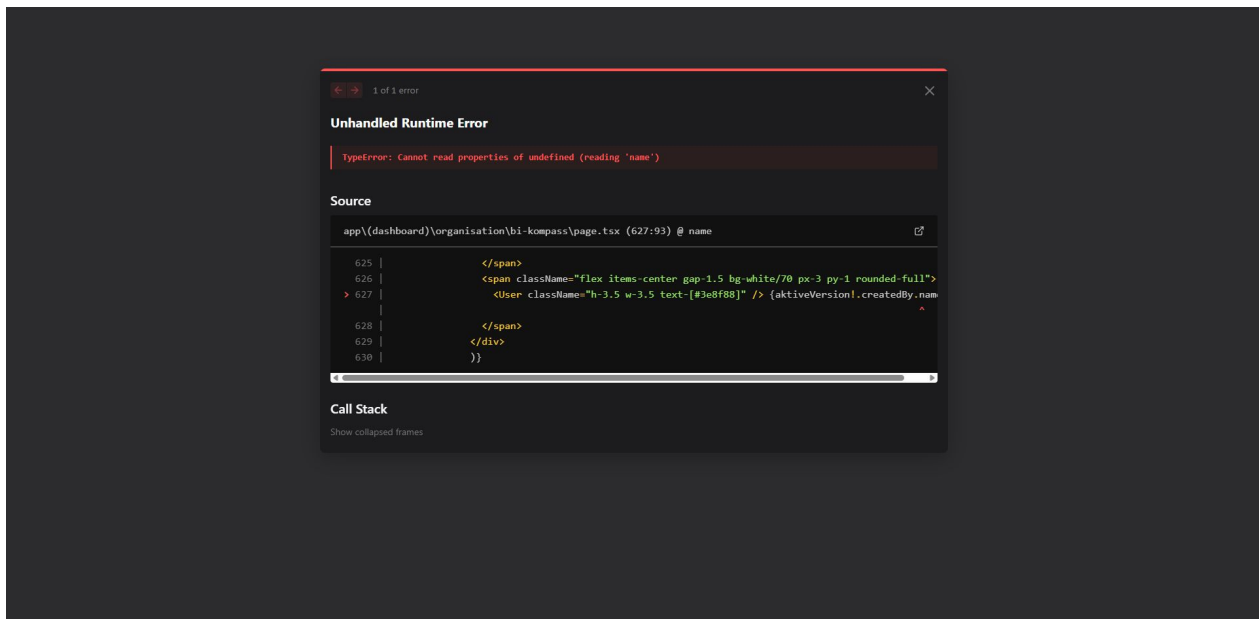
Priority: High. Reproduced populated-page crash.

Observed behavior. Opening BI-Kompass with an active version saved in PostgreSQL triggers a client exception at `aktiveVersion.createdBy.name`. The response does not contain the assumed nested author object.

Consequence. The populated module cannot be used to inspect or maintain versions.

Recommended correction. Define and load the author projection deliberately, handle a missing/deleted author safely, and add a page-level recovery state.

Acceptance check. Create two versions through the real API, activate each, reload the page and edit/delete a disposable version through the UI. Test missing authors and an empty version collection.



Screenshot evidence. WEB-013. BI Compass crashes on the active version author field with a real persisted version.

UI-27 | Circle editing submits a different contract from the persisted circle model

Priority: High. Reproduced form/API failure.

Observed behavior. A circle created through Brave can be read, but its edit form sends purpose/purposeDetails/verantwortlichkeit and empty date strings rather than a validated circle write shape. The reproduced PATCH returns HTTP 500 with a null Delta path.

Consequence. An apparently editable circle cannot save the tested change.

Recommended correction. Map only supported fields into a typed update DTO, convert optional dates to the agreed null/date representation and validate invalid binding before calling Delta.Patch.

Acceptance check. Edit and clear every displayed field, save, reload and compare the database-backed values.

The screenshot shows a web application interface for editing a circle. The sidebar on the left contains navigation links: Mail-Verteiler, BI-Guide, BI-Kompass, Organigramm, Life Cycle Prozess, Kreise, Spannungen, Hilfe, Dokumentation, and Einstellungen. The main form has the following fields:

- Name:** FT Browser Circle edited
- Gültig ab:** mm dd/yyyy
- Gültig bis:** mm dd/yyyy
- Zweck:** Created through visible Brave with local PostgreSQL.
- Verantwortlich für:**
- Hierarchie:** Dieser Kreis ist aktuell ein Root-Kreis. Er kann an einen anderen Kreis angehängt und danach als Subkreis geführt werden. Vorhandene Subkreise werden dabei mit verschoben.
- Ziel-Kreis:** Ziel-Kreis wählen
- Buttons:** Anhängen
- Summary:** 0 Rollen, 0 Meetings, 0 Spannungen, 0 Entscheid., 0 Subkreise
- Lebenszyklus-Historie:** Aktiv seit 08.09.2026

Screenshot evidence. WEB-018. Circle editing remains in the failed save state. The paired contract probe records the invalid binding and HTTP 500.

UI-28 | Meeting creation and detail queries disagree with the API

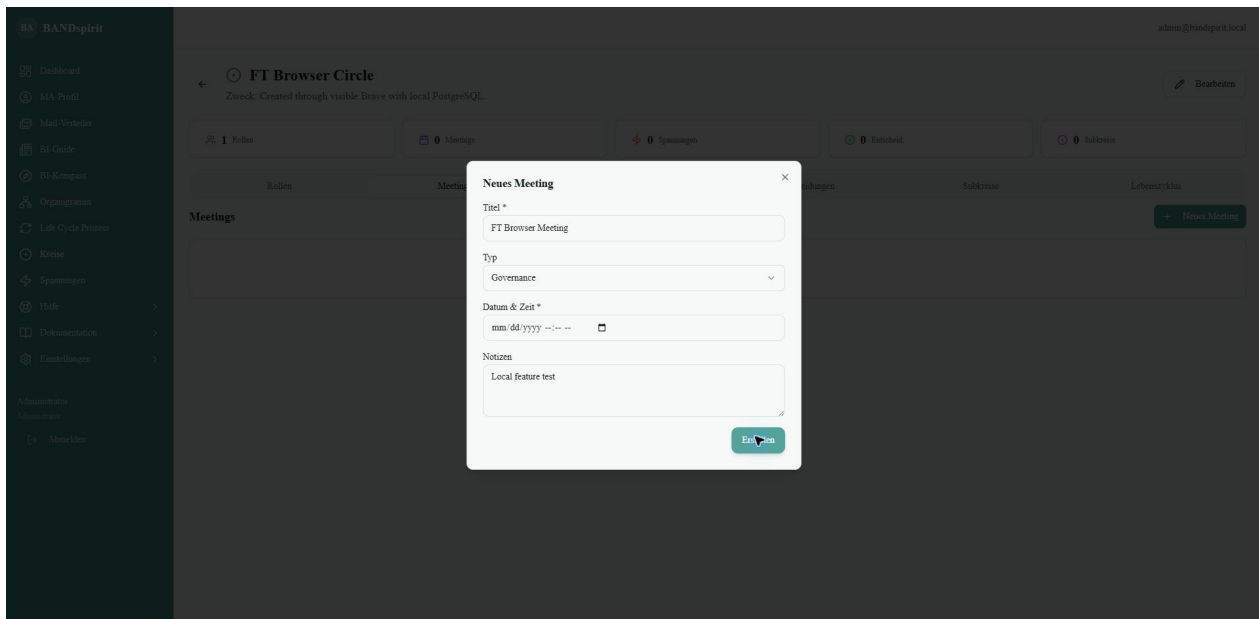
Priority: High. Reproduced unavailable workflow.

Observed behavior. The form sends title, meetingType and notes to a model with different declared fields, producing HTTP 400. A known persisted meeting cannot be opened: the detail loader requests unsupported Proposals/Objections/Decision navigations, gets 400 and redirects to Organisation.

Consequence. Meeting creation, navigation and dependent proposal/objection/decision UI actions are blocked.

Recommended correction. Agree on meeting list/detail/write DTOs and supported navigation names. Load actual meetings instead of an empty adapter array, map the form fields, and preserve a useful error state on the detail page.

Acceptance check. Create a scheduled meeting from a circle, reload it, change status, add an agenda proposal, raise/resolve an objection and save a decision. Read the persisted state independently after each step.



Screenshot evidence. WEB-022. The meeting form is present, but the displayed request contract does not save to the backend. The detail route is independently blocked by its invalid expansion query.

UI-29 | The user deactivation action deletes the account

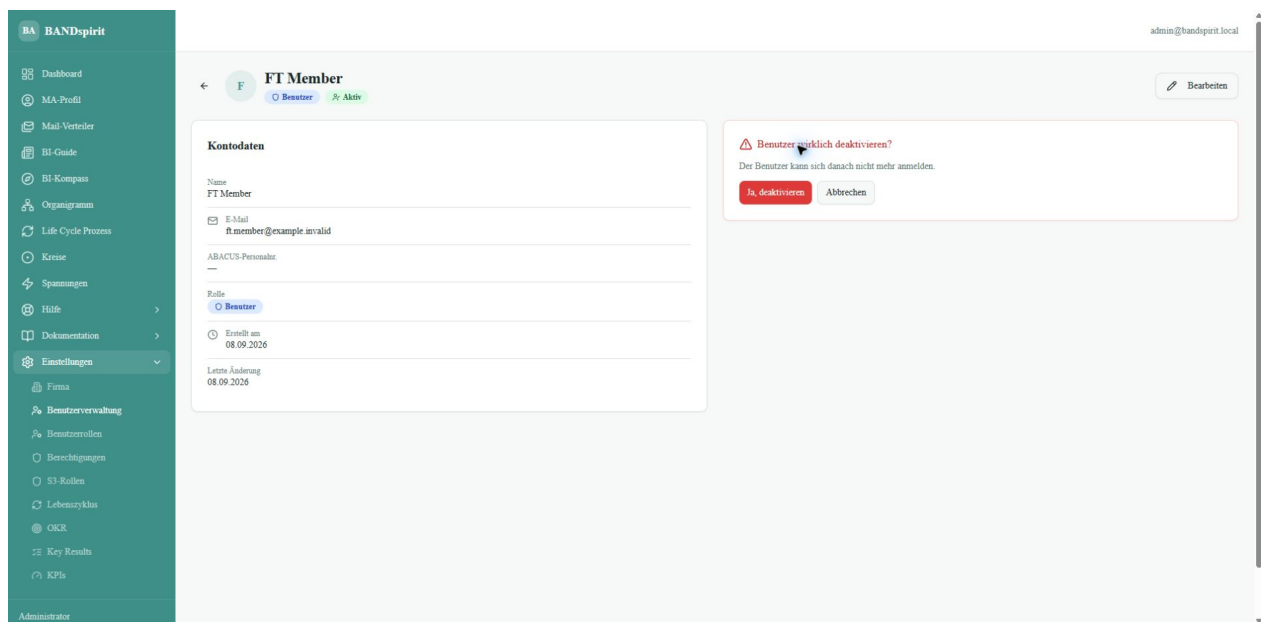
Priority: High. Reproduced destructive semantic mismatch.

Observed behavior. The action and confirmation say Benutzer deaktivieren and explain that the account will no longer be able to log in. The handler instead sends DELETE /odata/Users(id), whose controller removes the user.

Consequence. An administrator expecting a reversible suspension can permanently remove an account and affect its related records.

Recommended correction. Implement deactivation as PATCH aktiv=false and apply the session-invalidation policy in APP-02. Keep permanent deletion as a separately named operation with an accurate consequence summary.

Acceptance check. Deactivate a disposable user and verify its ID, history and memberships remain while login and existing sessions are blocked.



Screenshot evidence. WEB-073. Confirmation promises deactivation, not permanent removal. This capture was cancelled; the earlier disposable-user deletion in WEB-031 establishes the actual operation.

UI-30 | The lifecycle process overview calls an unimplemented endpoint

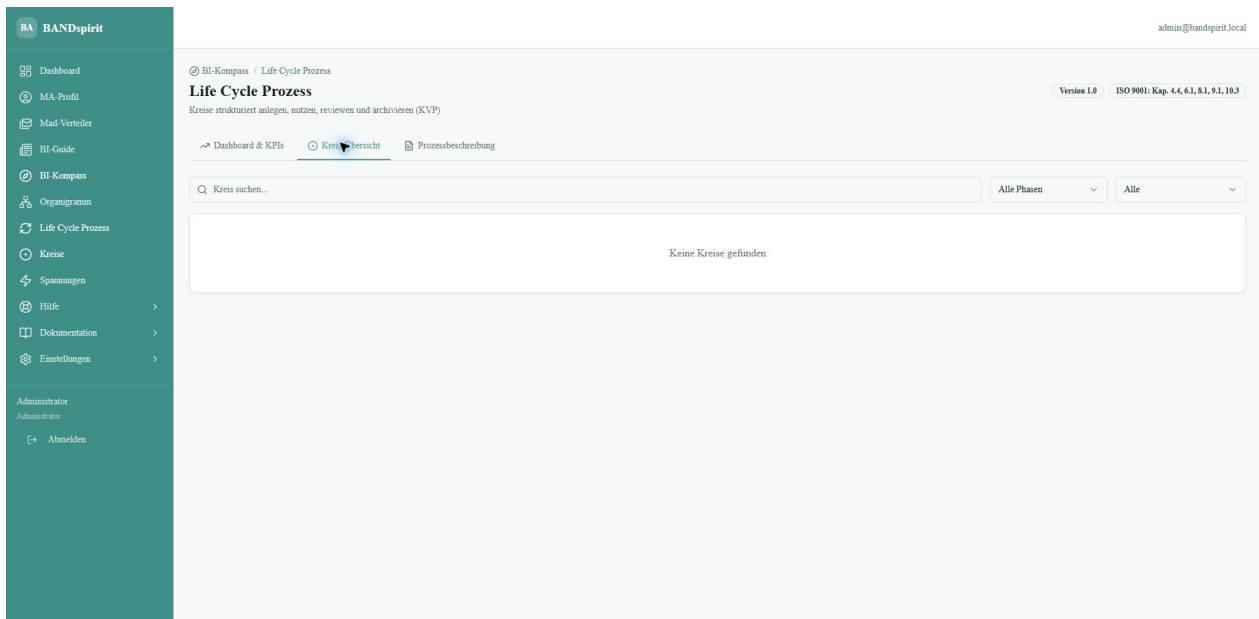
Priority: High. Reproduced missing backend integration.

Observed behavior. The guide text renders, but its circle-data request to `/api/org/circle-lifecycle` returns HTTP 404. No matching controller was found in the supplied backend.

Consequence. The process overview, review and phase/archive actions that depend on its data cannot be completed.

Recommended correction. Implement a documented, authorized process API or adapt the page to existing supported endpoints. Distinguish loading, no circles and unavailable integration.

Acceptance check. Populate circles in different lifecycle phases and review states. Verify overview counts, filters, review submission and allowed transitions against persisted history and current phase.



Screenshot evidence. WEB-054. The lifecycle process overview is empty despite three stored circles because its endpoint returns 404.

UI-31 | Password recovery forms use incompatible request and response fields

Priority: High. Source-confirmed authentication contract mismatch.

Observed behavior. The forgot-password UI requires data.success, while the backend returns nachricht on HTTP 200. Reset sends password, but the required API field is newPassword; it also expects success/error rather than nachricht/fehler.

Consequence. A successful recovery request can look failed, and the displayed reset form does not satisfy the backend write contract.

Recommended correction. Use shared typed authentication DTOs and status-based success handling with an agreed response envelope. Keep NextAuth routes and backend authentication actions explicitly routed to their intended handlers.

Acceptance check. After email checks are in scope again, verify the complete recovery flow.

UI-32 | A direct user-detail visit can redirect before the session is ready

Priority: Medium. Reproduced session-initialization race.

Observed behavior. Opening a known user-detail URL directly redirected to the user list, while opening the same record through in-app navigation loaded its details. fetchUser runs when id changes without waiting for session, and the effect does not rerun when session becomes available. The catch path treats any failure as Benutzer nicht gefunden.

Consequence. Reloaded or shared user-detail links can appear broken even though the record exists and the signed-in administrator is authorized.

Recommended correction. Wait for session status to resolve before requesting protected data, include the session dependency, and cancel stale requests.

Acceptance check. Open and reload the same detail URL with a warm and cold session, then

navigate from the list. All authorized paths must load the same record; missing and forbidden IDs need distinct messages.

4 Database and migration findings

DB-01 | Business identifiers lack referential integrity

Priority: High. Confirmed schema gap.

Observed behavior. No foreign keys link PasswordResetTokens.UserId, S3Objections.UserId, SupportTickets.ErstellerId, S3Proposals.MeetingId, the parent/child IDs of S3CircleLinks, or the circle/definition IDs of MailVerteiler to their apparent parents.

Consequence. These ordinary business references can point to missing records.

Recommended correction. Profile orphans, map incompatible identifiers and add the intended foreign keys with deliberate nullability and delete policies.

Acceptance check. Attempt invalid references in a disposable PostgreSQL database and confirm rejection.

DB-02 | Application-role membership uses editable names

Priority: High. Confirmed identity-model defect.

Observed behavior. Users.Role and RolePermissions.Role store the application role name as text without a foreign key to BenutzerRollen. Ordinary role rename updates the catalog row without migrating those references in the inspected path.

Consequence. Renaming a role can strand membership or grants.

Recommended correction. Migrate memberships and grants to a stable role ID using an explicit name-to-ID mapping. Preserve names for display and verify authorization before removing legacy columns.

Acceptance check. Rename an ordinary role that has users and custom permissions. Membership and effective grants must remain unchanged; unknown mappings must be rejected or resolved explicitly.

DB-03 | Circle/definition uniqueness exists only in one create path

Priority: High. Confirmed missing business constraint.

Observed behavior. Role creation checks whether the same CircleId/RollenDefinitionId pair exists. The database has separate indexes but no unique index for the pair, and generic PATCH does not perform the same check.

Consequence. Concurrent inserts or a later update can create duplicate circle-role instances even though the create workflow says the combination must be unique.

Recommended correction. Confirm the business invariant, reconcile any existing duplicates and add a composite unique constraint. Retain friendly API validation and translate database uniqueness failures into clear conflict responses.

Acceptance check. Test two concurrent creates and a PATCH that collides with another instance. Only one logical instance should remain, and the API should return a controlled conflict.

DB-04 | Four detail tables allow zero or two owners

Priority: High. Confirmed ownership ambiguity.

Observed behavior. Role responsibilities, metrics, checklists and documents each have optional RoleId and RollenDefinitionId fields. Both may be null or both populated.

Consequence. Rows can be orphaned at the business level or owned inconsistently by a definition and an unrelated circle-role instance.

Recommended correction. Choose a definition-owned, instance-owned or explicit override model. If two nullable owner columns are retained during transition, enforce the chosen combination with a CHECK after repairing existing rows.

Acceptance check. Exercise neither-owner, both-owner, nonexistent-owner and valid-owner cases through all write paths. Ensure row visibility follows the chosen ownership policy.

DB-05 | The definition-owner migration does not backfill legacy details

Priority: High. Confirmed incomplete data transition.

Observed behavior. The migration adds definition owner columns and makes instance ownership optional without translating existing rows. Frontend detail queries now use RollenDefinitionId in the inspected settings page.

Consequence. Instance-only historical records can remain in the database yet disappear from the definition-based view.

Recommended correction. Design an explicit backfill and conflict policy. Several instances can share one definition, so assigning all rows to the definition can create duplicates or accidentally share local content.

Acceptance check. Seed different legacy details under two instances of the same definition, apply the migration and verify every row remains intentionally visible, retained or flagged for reconciliation.

DB-06 | Responsibilities have three editable storage locations

Priority: Medium. Confirmed competing representations.

Observed behavior. Responsibilities can live in S3Roles.Responsible, S3RollenDefinitionen.Verantwortlichkeit or S3RolleVerantwortlichkeiten rows. The import fills the definition text for 185 definitions, leaves the 162 role-instance Responsible values null and inserts no responsibility-detail rows.

Consequence. Different readers can display different content or an empty result, even when another representation contains the information.

Recommended correction. Select an authoritative representation and define any explicit local override. Reconcile and backfill existing content before retiring a field or switching every reader.

Acceptance check. Populate each legacy location with distinct synthetic content. Confirm migration retains all meaningful content and the UI resolves it by the documented precedence rule.

DB-07 | Repeated role names have no explicit template/variant semantics

Priority: Medium. Business-design decision.

Observed behavior. The import contains 185 definitions, including 20 each named Lead Link, Moderator:in and Dokumentator:in. Some same-named definitions have different text.

Consequence. The catalog is difficult to interpret without knowing whether these are independent roles, shared templates or local variants.

Recommended correction. Have the domain owner choose independent definitions or shared templates with explicit overrides. Display circle/domain context where needed and merge only after comparing responsibilities, purpose and usage.

Acceptance check. Review every proposed merge with a content comparison and reversible ID mapping. Demonstrate that editing a template changes exactly the intended circles.

DB-08 | Validity fields do not define membership history

Priority: Medium. Confirmed policy/constraint gap.

Observed behavior. All entities inherit optional DateFrom and DateTo without a declared ordering check. The unique UserId/RoleId assignment pair applies across all dates, so separate repeat-tenure rows for the same person and role cannot coexist.

Consequence. Invalid date ranges can be stored.

Recommended correction. Define interval boundaries, overlap rules, time zone and current-versus-history behavior. Add range checks and, if repeated tenures are required, introduce a suitable history model and uniqueness/exclusion strategy.

Acceptance check. Test reversed and open intervals, adjacent tenures, overlapping tenures and a person leaving then returning to the same role. Also verify read queries apply the intended temporal policy.

DB-09 | VALIDATION PASSED covers fewer checks than the script prints

Priority: High. Confirmed validation gap.

Observed behavior. The validator prints multiple counts and diagnostics, but its final assertions cover only three organizational counts, orphan assignments and duplicate assignment pairs. Other printed diagnostics do not necessarily cause failure.

Consequence. A successful final message can be mistaken for full source reconciliation or complete integrity validation.

Recommended correction. Separate informational diagnostics from mandatory assertions. Make required invariants fail the run, document expected seeded-plus-imported totals and include semantic source reconciliation.

Acceptance check. Introduce one controlled defect for each mandatory check in a disposable copy.

DB-10 | Import transformations and onboarding decisions are not fully documented

Priority: High. Confirmed reconciliation gap.

Observed behavior. The workbook has 309 members versus 308 imports, with one blank-email row omitted. All source Suspended values are false, but only 20 imported users are active and 288 inactive.

Consequence. Counts alone cannot explain dropped, split or remapped records.

Recommended correction. Produce a row-level source-to-target ledger with imported/transformed/excluded outcomes and reasons. Document seeded prerequisites and the activation/password or SSO onboarding process.

Acceptance check. Account for every source record and obtain owner sign-off on exclusions and activation rules. Verify a representative user can complete the intended onboarding flow.

DB-11 | Organizational and global measurement models are disconnected

Priority: Medium. Business-design decision.

Observed behavior. OKRs and KpiDefinitions have no circle/owner foreign key. Role-specific metrics are represented separately and are not linked to a global KPI definition.

Consequence. If reports require circle accountability or shared metric identity, names must not be used as an implicit join.

Recommended correction. Define which metrics/objectives are global and which are owned. Add explicit owner/definition associations for required reporting while retaining legitimate global records.

Acceptance check. Trace a metric from its definition through role/circle ownership and measurements. Renaming it must not change identity, and reports must distinguish shared from local measures.

DB-12 | Company singleton exists only as an application convention

Priority: Low. Confirmed convention without enforcement.

Observed behavior. The controller addresses the text ID singleton and resets that ID during PATCH. The table primary key ensures unique IDs but does not prevent rows with other IDs.

Consequence. Another writer can create additional company records, and concurrent first-time requests can compete to create the singleton.

Recommended correction. If one company row is the real invariant, enforce the allowed singleton key and initialize it idempotently.

Acceptance check. Attempt a second company ID and concurrent initialization. Require the agreed cardinality and a controlled response rather than an unhandled duplicate-key failure.

DB-13 | Email comparison and canonicalization are not explicit

Priority: Medium. Identity-policy decision.

Observed behavior. Account lookup and signup compare the supplied email string directly. The reviewed paths do not normalize whitespace/case, and the snapshot specifies a unique text column rather than an explicit canonical identity key.

Consequence. An identity provider may consider two variants equivalent while BANDspirit compares them as different strings.

Recommended correction. Define a canonical identity policy and apply it consistently to local signup, login, imports and SSO linking.

Acceptance check. Test approved case/whitespace variants and SSO/local linking against PostgreSQL. Review any existing collisions manually before adding a normalized unique key.

DB-14 | Application startup performs schema migration

Priority: Medium. Confirmed deployment coupling.

Observed behavior. The startup path includes Database.MigrateAsync. Schema changes are therefore coupled to application startup when that path is enabled.

Consequence. Multiple instances starting together can compete over schema changes; application startup may require schema privileges and may block on migration work.

Recommended correction. Deploy reviewed migrations once through a controlled job using appropriate credentials, generated SQL and a tested recovery plan.

Acceptance check. Rehearse an upgrade from a representative previous version, including two application startups and a failed migration. Verify only the designated migration job changes the schema.

DB-15 | Deletion and retention policies are implicit in cascades and cleanup

Priority: Medium. Business-design decision.

Observed behavior. Many dependents cascade on deletion, while hierarchy/catalog references use Restrict. Cleanup defaults include daily execution, 90-day app-log retention and removal of sufficiently old used/expired reset tokens.

Consequence. A permitted parent deletion may remove memberships or governance detail that a stakeholder expected to preserve.

Recommended correction. Agree retention, deletion previews, soft-delete/history needs and recovery objectives by domain. Validate each cascade and coordinate database/object-storage recovery; preserve security-safe audit evidence.

Acceptance check. Restore a backup with matching objects and run deletion scenarios for circles, roles, definitions and people. Compare the lost/preserved records with the signed-off policy.